CSC8204 Secure Software Development

Name: Aniket Vinod Nalawade

Student No: 250535354

Part A: Applied Risk Analysis

1. **Business Goal's of Praxis (Rank which is using NIST Classification)**

ID	Business Goal	Description	NIST Category	Rank
G1	Demonstrating the feasibility of higher-assurance secure system development	Show that the EAL5+ style formals methods can be practically attain in industry	Mission Goal	1
G2	Display the Praxis Correctness-by-Construction methodology	Provide proof of evidence that CbC provides a fault-free, accurate, efficient process	Strategic Goal	2
G3	Authorizes NSA to disseminate formal practice to industry	Craft the evidence usable by wider defence or industry partners	Market Influence Goal	3
G4	Displays economical of rigorous development vs traditional approaches	Harvest metrics, defect counts, formal methods overheads	Financial Goal	4
G5	Bring a working demonstrator (Tokeneer ID Station)	Building an functioning of the high assurance security system	Operational Goal	5

2. Business Risks

Risk ID	Business Risk	Indicators	Likelihood	Impact	Severity	One-Sentenced Rationale
B1	Failed in the demonstration feasibility of accurate high-assurance development	After delivery defects; formal proof of failures; missing EAL5 assurance	Medium	High	High	Inhibit the core purpose of the project for NSA.
B2	Cost or schedule overflows undermining cost-effectiveness asserts	Un-used milestones, excessive alter, high person day usage	Medium-high	Medium-High	High	Weaken's the project claims that is CbC cost-effective.
B3	Restricted industry uptake of the results	Negative stakeholder review, insufficient documentation clarity	Medium	High	High	Deforms NSA's goal of making wider contractor practices.

3. Technical Risks (McGraw's Methodology)

Risk ID	Technical Risk	Description	Likelihood	Impact	Controls Mentioned (Tokeneer/ Principles)	Rationale
TR1	Designing the flaws in formal specification	Obscurities or inconsistencies in the large Z specs	Medium	High	Proof of obligations, manual examination, schema reviews	An flawed specificatio cascades into the fals design and code

TR2	Weakening models of external environment(drivers, sensors ,libraries)	Majority of the components simulated rather than officially modelled	Medium	High	Principles, stubs, model-based testing	Replica hid real world behavior conflicts.
TR3	Partial coverage of security properties	EAL5 needs sample proofs, not full-system formal proofs	Medium	Medium-High	Selected proof challenges, partial verification	Incomplete proofs can leave unverified vulnerabiliti
TR4	Variable refinement between Z spec and SPARK implementation	Behavioral clashes during refinement to code phase	Medium	High	SPARK static analysis and refinement checks	Refinement error break correctness by-constructio
TR5	Limited penetration testing due to the simulated interfaces	Mislead real hardware testing reduces the chances of the real world assurance	High	High	Restricted adversarial testing, mock hardware	Real-world attacks may avoid simulated protections.

4. Risk Synthesis

Business Goal	Business Risk	Technical Risks Contributing	Explanation
G1: Feasibility of higher-assurance system	B1	TR1, TR3, TR4	Wrong specifications or incomplete formal proof threaten feasibility claims.
G2: Demonstrate the CbC strength	B2	TR1, TR2, TR4	Pitfalls create mostly alter and undermine methodology strength.
G3: Dissemination of the industry	B3	TR2, TR5	Weakened modelling and weakened pen testing limits the probability to other partners

G4: Cost-effectiveness	B2	TR1, TR3	Altering from specification or proof gaps increases the costs
G5: Working demonstrator	B1,B3	TR2, TR5	Hardware and environment points the risk demonstrator reliability.

5. Mitigations

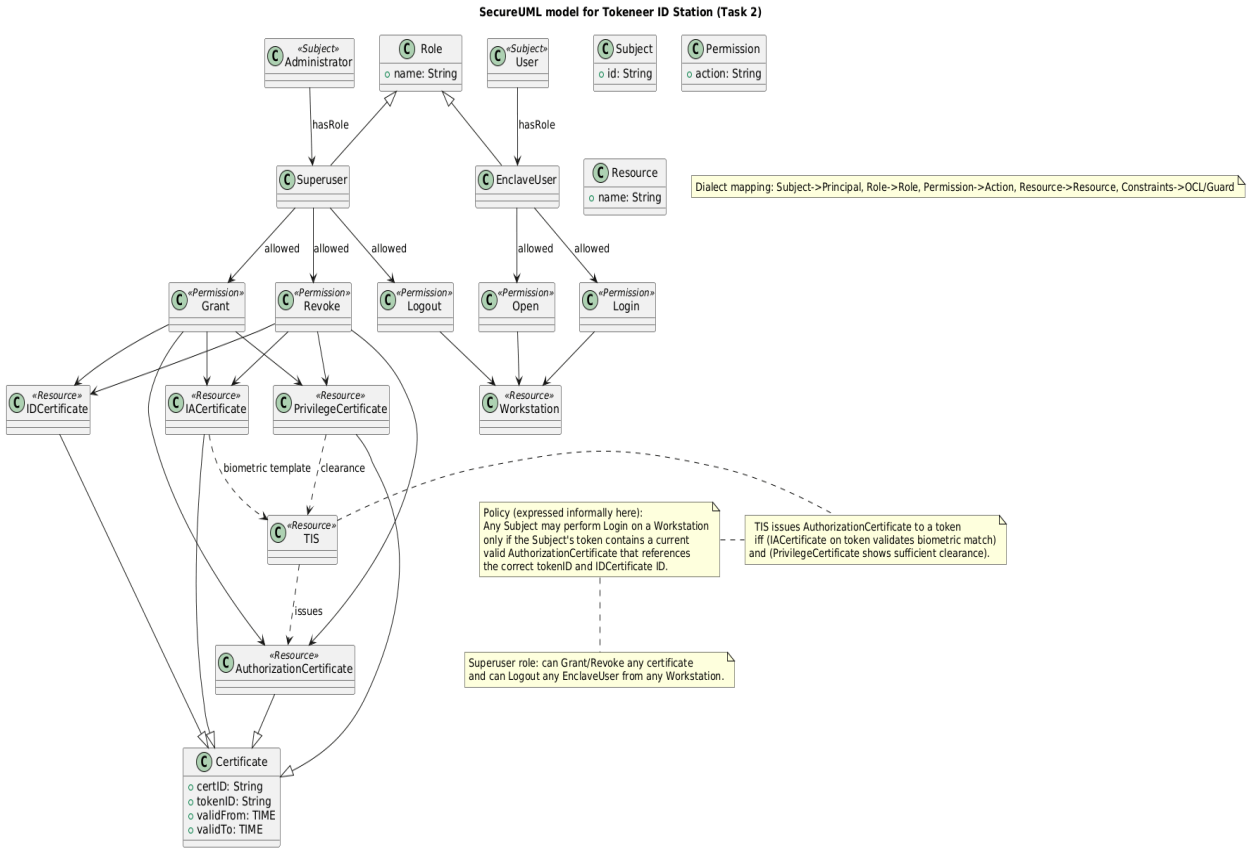
Tech Risks (ID)	Mitigation Strategy
TR1	Strengthening the formal specification reviews and add more proof of obligations; multi-view architecture inspection.
TR2	Rise in fidelity of environment simulation; presents the explicit environment contracts and better off hardware modelling.
TR3	Extend the verification scope from the sample proofs of full property coverage; adding the property-based testing.
TR4	Applying the strong SPARK static analysis and command retrieve relations for Z-SPARK refinement.
TR5	Conducting the real hardware penetration testing, fuzzing and adversarial fault-inject experimenting.

6. Rationale

The reason behind this risk analysis is based on the risk management methodology applied by McGraw who argues that it must encompass a systematic succession of organisational objectives to the business risks and ultimately the technical risk to which business risks are exposed. The objectives of the business were based on the direct Tokeneer project documentation, which involved the NSA contracting Praxis to prove the feasibility, cost-efficiency, and commercial relevance of high-assurance development based on Correctness-by-Construction (CbC). These objectives have been categorised as NIST business goal hierarchy to ascertain a proper prioritisation of mission-critical, strategic, market-oriented, financial and operational objectives.

The selected business risks were identified due to their realism and the largest impact on the accomplishment of the following objectives, the ineffective demonstration of feasibility, the overspending of costs or time span, and the decreased uptake by the industry. All these risks can be directly associated with the explicit statements in Tokeneer report regarding the project purpose and anticipated outcomes. The technical risks were selected according to real world engineering issues as presented throughout the Tokeneer and Principles of Software Security documents, such as vulnerabilities of formal specification, incomplete modelling, partial proof obligations, refinement errors and lack of sufficient penetration testing. These risks match the touchpoints of McGraw, including architectural analysis, code review and penetration testing making sure that the analysis does not contradict the accepted practices in developing secure systems methodologically. This synthesis shows that there is a full sequence of influence in which every technical risk poses a threat to one or more business risks, which subsequently endangered certain business objectives. This interconnected mapping provides completeness, logical coherence, and traceability which is an important element of a quality risk analysis. The chosen mitigation measures are based on the best practices of McGraw and the Tokneer engineering situation which provides high impact and realistic interventions. Altogether, the reasoning reflects a consistent, evidence-based method of listing and dealing with security risks because it provides the correspondence to the academic framework provided in the module as well as to the realities present in the Tokeneer system.

Part B SecureUml RBAC Design



SecureUml Diagram

Rationale Summary:

This model of SecureUML is simply a topical picture of the role-based access control of the Tokeneer Identification Station (TIS). We distinguish between the principle bits: Subjects (the users and admins), Roles (EnclaveUser, Superuser), Permissions (Grant, Revoke, Open, Login, Logout) and Resources (Workstation, TIS itself and the types of different certificates). The certificate classes are a derived tree that derives out of a general Certificate class, and thus can be connected with the things we actually need in the Tokeneer policy e.g. tokenID or certificate IDs. In fact, we put the security policy in three places: (1) who does what, (2) some ad hoc, but incisive notes which are, later to become OCL pre/post conditions, and (3) the place where the TIS issues an Authorization Certificate after determining that a user has been authenticated (by biometrics) and cleared. The intent is that the Superuser role is a powerful role: it is able to Grant, and Revoke on all certificate resources and even kick out any EnclaveUser just as the need to kick out superusers manage certificates and end sessions. The rule of logins is a guard: a user is allowed to log

in SecurityGuard a Workstation only when he/she possesses an up-to-date valid AuthorizationCertificate which links to the right token and IDCertificate.

This figure serves as the input of the secure UML of a security workflow model. Virtually the class/role/permission thing simply is projected to elements of the metamodel of SecureUML, and those informal notes can be converted to OCL constraints (or SecureUML specific blocks) prior to code generation. The design is in such a way that is clear to formal verification and traceability: all the linkage fields (tokenID and certificateIDs) are arranged in a way that allows the Dafny and SPARK exercises later in the course to utilize them, and makes the runtime tests that the TIS policy requires easily understandable. Ultimately, the authentication flow remains simple (biometric check > AuthorizationCertificate issuance > Workstation login guard) with the model, which is excellent in performing risk analysis and enables to automatically enact with role/should-check enforcement during execution.

Part D Formal Verification

1. Sums Method

Goal: Determining the weak requirement that completes the final condition m > n after executing the following sequence:

m := x

n := y

a := 2*m + n

n := n - 1

m := a

To calculate the weak requirement, we work reverse through the assignments, swapping the right-hand side of each assignment into the condition that follows it.

Step-by-step Reasoning

Previous statement: m := a

To make sure m > n after this line, we must require:

Replace m with a → a > n

Next: n := n – 1

For a > n to hold after this update, substitute n - 1 for n in the condition:

a > (n - 1)

Next: a := 2*m + n

Substitute 2*m + n in place of a in a > (n - 1):

2*m + n > n – 1

Simplify:

2m > –1

Next: n := y

Since n does not appear in 2m > –1, this statement does not affect the condition:

Before the condition remains 2m > –1

First statement: m := x

Replace m with x:

2x > –1

Final Weak Precondition:

2x > –1

This is the least restrictive requirement on the initial state that ensures the after the condition m > n is true after the program runs.

2. UpdateAlarms Method

Goal: Find the weak requirement for the after condition :

Q:

(alarm == alarming) ↔ (doorAlarm == alarming ∨ auditAlarm == alarming)

The code:

```
if (doorAlarm == alarming || auditAlarm == alarming)
    alarm := alarming;
else
    alarm := silent;
```

Let the condition

C ≡ (doorAlarm == alarming) ∨ (auditAlarm == alarming).

The after condition can be restated as:

alarm is alarming exactly when C is true.

We use the standard wp rule for conditional:

wp(if B then S1 else S2, Q) = (B → wp(S1, Q)) ∧ (¬B → wp(S2, Q))

Here, B = C.

Then-branch: alarm := alarming

Exchange alarming for alarm in Q:

(alarming == alarming) ↔ C

True ↔ C

This simplifies directly to C.

Else-branch: alarm := silent

Exchange silent for alarm:

(silent == alarming) ↔ C

False ↔ C

Equivalent to ¬C.

Apply wp rule:

(C → C) ∧ (¬C → ¬C)

Each implication is always true.

Final Weak After Condition:

True

Meaning: the code always establishes the after condition regardless of the initial values of the variables.